Eventtra

FYP Group

Awais Shahid	**19F-0193**
Fatima Ijaz	**19F-0185**
Muhammad Saad	**19F-0220**

Supervised by

Mr. Tahir Farooq

Department of Computer Science

National University of Computer and Emerging Science Chiniot-Faisalabad Campus, Pakistan

2023

Anti-Plagiarism Declaration

This is to declare that the above publication produced under the:

Title: _____

is the sole contribution of the author(s) and no part hereof has been reproduced on as it is basis (cut and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. I/We will be responsible and liable for any consequence if violation of this declaration is determined.

Date: _____

Student 1

Name: Awais Shahid.

Signature: _____

Student 2

Name: Fatima Ijaz.

Signature: _____

Student 3

Name: Muhammad Saad.

Signature: _____

Abstract

Eventtra will provide a platform for event management. It will also target social anxiety attendees to experience events using VR Headset which will stream over 360-degree camera. It will connect Admin, Organizers and Attendees. Admin will create description page and set hierarchy. Organizers will manage events along with multiple features. Communication rooms for each event will be created for organizational communication. Attendees will get real-time notifications for the events. Attendees can enroll and pay fees online for events. Attendees will have the option to give review for the attended event. Attendees will get online certificates for attending the event.

Table of Contents

1. INTRODUCTION 1

2. VISION DOCUMENT 1

2.1. PROBLEM STATEMENT 1

2.2. BUSINESS OPPORTUNITY 1

2.3. OBJECTIVES 2

2.4. SCOPE 2

2.5. CONSTRAINTS 2

2.6. STAKEHOLDERS AND USER DESCRIPTION 2

2.6.1. Market Demographics 2

2.6.2. Stakeholder Summary 3

2.6.3. User Environment 4

2.6.4. Stakeholder Profile 4

2.6.4.1. Supervisor of the Project 4

2.6.4.2. Development Team of the Project 5

3. SYSTEM REQUIREMENT SPECIFICATION 6

3.1. SYSTEM FEATURES 6

3.2. FUNCTIONAL REQUIREMENTS 6

3.2.1. Register/Login 6

3.2.2. Update profile 6

3.2.3. Event creation 6

3.2.4. Hierarchy Management 6

3.2.5. Sub-event management 6

3.2.6. Event enrollment 6

3.2.7. Event Requests 7

3.2.8. Online Streaming 7

3.2.9. Runtime notifications 7

3.2.10. Send/receive messages 7

3.3. NON-FUNCTIONAL REQUIREMENTS 7

3.3.1. Responsive Layouts 7

3.3.2. Security Requirements 7

3.3.3. Scalability 7
3.3.4. State Management 7
3.3.5. Availability 7
4. USE CASE DIAGRAM 8
5. HIGH LEVEL USE CASE 9
5.1. REGISTER / LOGIN 9
5.2. AUTHENTICATION 9
5.3. UPDATE PROFILE 9
5.4. CREATE EVENTS 9
5.5. SEND / RECEIVE MESSAGES 10
5.6. VIEW ONLINE STREAM 10
5.7. REQUEST FOR EVENT 10
6. EXPANDED USE CASE 11
6.1. SET HIERARCHY 11
6.2. MANAGE SUB-EVENTS 11
6.3. SETUP ONLINE STREAM 12
6.4. GENERATE RUNTIME NOTIFICATIONS 12
6.5. ENROLL FOR EVENT 13
7. DOMAIN MODEL 14
8. CLASS DIAGRAM 15
9. SWIMLANE DIAGRAM 16
10. SYSTEM SEQUENCE DIAGRAM 17
10.1 SET HIERARCHY 17
10.2 MANAGE SUB-EVENTS 18
10.3 SETUP ONLINE STREAM 19
10.4 GENERATE RUNTIME NOTIFICATIONS 19
10.5 ENROLL FOR EVENT 20
11. SEQUENCE DIAGRAM 21
11.1 ADMIN 21
11.2 ORGANIZERS 22

11.3 ATTENDEE 23

12. DATA FLOW DIAGRAMS 24

12.1. LEVEL 0 DATA FLOW DIAGRAM 24

12.2. LEVEL 1 DATA FLOW DIAGRAM 25

12.3. LEVEL 2 DATA FLOW DIAGRAM 26

12.3.1. Event Creation: 26

12.3.2. Sub-Event Management: 27

12.3.3. Online Stream: 28

12.3.4. Event Enrollment: 29

13. PACKAGE DIAGRAM 30

14. DATABASE SCHEMA 31

15. COMPARISON 32

16. UNIT TEST PLAN 33

16.1 INTRODUCTION 33

16.2 TEST ENVIRONMENT 33

16.3 TEST APPROACH 33

16.4 TEST REPORTING 33

16.5 TEST DELIVERABLES 33

16.6 TEST MANAGEMENT 33

16.7 UNIT TEST REPORT 34

16.7.1 Module: addEventdetails.java 34

Unit: testValidDate1() 34

Unit: Unit: testInvalidDate1 34

16.7.2 Module: adminEventRequests.java 35

Unit: dataValid1() 35

Unit: dataInvalid1() 35

16.7.3 Module: MyEvent.java 36

Unit: MyEventValid1() 36

Unit: MyEventInvalid1() 36

16.7.4 Module: MyUser.java 37

Unit: MyUserValid1() 37

Unit: MyUserInValid1 () 37

16.7.5 Module: PaymentInfo.java 38

Unit: PaymentValid1() 38

Unit: PaymentInvalid1() 38

16.7.6 Module: subEventRequests.java 39

Unit: subEventRequestsValid1() 39
Unit: subEventRequestsNotValid1() 39

Tables

Table 1 Problem Statement 1

Table 2 Stakeholder Summary 3

Table 3 Supervisor of the Project 4

Table 4 Development Team of the Project 5

Table 5 Register/login High Level Use Case 9

Table 6 Authentication High Level Use Case 9

Table 7 Update Profile High Level Use Case 9

Table 8 Create Events High Level Use Case 9

Table 9 Send/Receive Messages High Level Use Case 10

Table 10 View Online Stream High Level Use Case 10

Table 11 Request For Event High Level Use Case 10

Table 12 Set Hierarchy Expanded Use Case 11

Table 13 Manage Sub-events Expanded Use Case 11

Table 14 Setup Online Stream Expanded Use Case 12

Table 15 Generate Runtime Notifications Expanded Use Case 12

Table 16 Enroll For Event Expanded Use Case 13

Table 17 Comparison 32

Table 18 addEventDetailsTest 34

Table 19 adminEventRequestsTests 35

Table 20 MyEventTest 36

Table 21 MyUserTest 37

Table 22 paymentInfoTest 38

Table 23 subEventRequests 39

Figures

Figure 1 Use Case Diagram 8

Figure 2 Domain Model 14

Figure 3 Class Diagram 15

Figure 4 Activity Diagram 16

Figure 5 SSD set hierarchy 17

Figure 6 SSD Manage Sub-events 18

Figure 7 SSD Setup Online Stream 19

Figure 8 SSD Generate Runtime Notification 19

Figure 9 SSD Enroll For Event 20

Figure 10 Admin Sequence Diagram 21

Figure 11 Organizer Sequence Diagram 22

Figure 12 Admin Sequence Diagram 23

Figure 13 Level 0 DFD 24

Figure 14 Level 1 DFD 25

Figure 15 Level 3 Event Creation DFD 26

Figure 16 Level 3 Sub-Event Management DFD 27

Figure 17 Level 3 Online Stream DFD 28

Figure 18 Level 3 Event Enrollment DFD 29

Figure 19 Package Diagram 30

Figure 20 Database Schema 31

1. Introduction

Eventtra will contain three entities i.e., Admin, Organizers and Attendees and will provide management services including a description page for events, hierarchy management and chat rooms for better communication. Diffident attendees will be able to experience events using VR Headset streamed on 360-degree camera. At the end attendees will receive certificates for their performance.

2. Vision Document

In this section, project vision is discussed in detail.

2.1. Problem Statement

Table 1 Problem Statement

Problem	1. Event management applications these days does not provide proper runtime notifications 2. These apps facilitate communication between attendees and the entire hierarchy on mobile devices. 3. In online streaming of events such as conferences and meetings, the audience lacks emotional connection just like they have when it is done physically because they are not provided with the proper experience.
Impact	1. Introverts prefer not to experience any event. 2. Attendee's queries cannot be solved at one place.
Solution	1. Organizers will provide runtime notifications. 2. Chat rooms will provide full communication between Admin, Organizers and Attendees. 3. Introverts will be able to experience events using VR Headset.

2.2. Business Opportunity

Different pricing plans will be able to help us target institutional as well as public attendees. As we will maintain a complete record of organized events, it will help us introduce similar events in the future. VR Headset will help engage more attendees. All these facilities will be centralized by our system.

2.3. Objectives

1. To create a platform with better communication between admins and users in an event
2. To create proper management of runtime notifications of events.
3. To provide a better experience of online streaming with 360-degree camera streaming.
4. To make a platform where meetings can be handled in private rooms.

2.4. Scope

Eventtra is an android application. The event management these days is scattered on different platforms i.e., WhatsApp is used for communication, zoom/google meet is used for online streaming and other apps are used for polling. So, we are aiming to make a centralized platform. Eventtra will help attendees to get more proper details of the events in which they are interested. Proper notifications will help them to keep complete track of the event. Chat rooms will help to resolve the queries. They will get event suggestions based on events attended or they are interested in so that they may not miss events of their interest. Also, there will be no hustle for registering and paying for an event

2.5. Constraints

1. If a 360-degree camera is not used with VR Headset, attendees will be forced to use a normal camera.
2. If specific organizers have no roles defined in the hierarchy, they won't be able to do anything in the system.
3. If the certificate template is not provided by the organizers for specific events, attendees will not get certificates for those events.

2.6. Stakeholders and User Description

The Stakeholder Profiles Attributes of the system are listed in this section.

2.6.1. Market Demographics

The faculty and students at FAST-NUCES are our target market. We are making this app to give people a better platform for signing up and receiving reminders to register for various events, as well as to make it simple for them to attend the event online and yet have a real-time experience. With every passing day, more people are using android phones and applications, and everything is moving from manual to automated processes. It can be utilized for different universities however for this project FAST NUCES is focused.

2.6.2. Stakeholder Summary

Table 2 Stakeholder Summary

Name	Description	Responsibilities
Eventtra Developers	It will include those who are involved in designing, developing and implementing of this project.	**1.** Should design android application considering user's needs. **2.** Proper testing should be Applied. **3.** Must use iterative model
End-User	End users will include Admin, organizers, and attendees.	**1.** Institutional users must login using their institute emails. **2.** Admins must define the roles of organizers. **3.** All users must have active internet connection.
Supervisor of project	Supervisor will be stakeholder of this product as he is involved in development process of this project with the development team.	**1.** Gives direction to development team. **2.** Supports cost saving development and deliver process improvements. **3.** Ensures that the system will be maintainable. **4.** Ensures that there will be a market demand for the product's features. **5.** Monitors the project's Progress. **6.** Ensures that project is in compliance with the documents generated during project planning and work products are properly delivered.

2.6.3. User Environment

Access to the app will be possible for any user with an internet connection. To install and run any application, users must be familiar with how to use an Android mobile device and other fundamental knowledge. For users to learn how to use the application without running into any issues, the user interface must be appropriate and installation instructions must be detailed enough. Since it is an android application, all the specified android phones and connected devices must be compatible.

2.6.4. Stakeholder Profile

2.6.4.1. Supervisor of the Project

Table 3 Supervisor of the Project

Representatives	Supervisor: Mr. Tahir Farooq
Description	He is involved in supervising activities of development process.
Type	He is technical stakeholders. He has expertise in domains which are being applied to this project i.e., android development, machine learning, web development.
Responsibilities	**1.** Gives direction to development team. **2.** Ensures that the system will be maintainable. **3.** Ensures that there will be a market demand for the product's Features. **4.** Monitors the project's progress. **5.** Ensures that project is following the documents generated during project planning and work products are properly delivered. **6.** They will facilitate development team to complete this project within specified resources.
Success Criteria	The completion of features which are being committed by development team at start of the project.
Involvement	**1.** Requirement reviewer **2.** Senior managers **3.** Reviews implementation
Comments	None

2.6.4.2. Development Team of the Project

Table 4 Development Team of the Project

Representatives	Awais Shahid Fatima Ijaz Muhammad Saad
Description	They are involved in designing and developing of application.
Type	They are technical stakeholders.
Responsibilities	**1.** Should develop this app considering user's needs. **2.** Proper testing should be applied to make it as effective as Possible. **3.** Proper data is required for better recommendations.
Success Criteria	Complete implementation of modules and features committed at the start of project by development team.
Involvement	**1.** Developers **2.** Testers
Deliverable	**1.** Documentation **2.** Mobile Application
Comments	None

3. System Requirement Specification

In this section, features and requirements of the system are explained.

3.1. System Features

1. Email Verification
2. Event History
3. Runtime Notifications
4. Responsive Layout
5. Online Certificates
6. Event Reminders
7. Custom Notification
8. Automatic Timed Notifications
9. Search Filtering
10. Independent Streaming Platform

3.2. Functional Requirements

The functional requirements of the system are listed below.

3.2.1. Register/Login

All users shall register and login to use Eventtra. Register and login will be authenticated.

3.2.2. Update profile

All users shall be able to update their profile when required. This will include changing password and all other information.

3.2.3. Event creation

Admin shall be able to create the main event in which the sub-events can be added later.

3.2.4. Hierarchy Management

Privileges of the hierarchy will be decided by the admin to ensure proper coordination. Admin will be able to provide his privileges to organizers if he wants to. Organizers will be able to gain access to events only after gaining privileges from admin.

3.2.5. Sub-event management

Organizers will be able to add sub-events in the main event. Organizers will be able to add pricing, polling options and certificates for the sub-event.

3.2.6. Event enrollment

Attendee will be able to enroll for the event. While enrolling for the event attendee may have to make votes and payments if required.

3.2.7. Event Requests

Students will be able to request for some events which can be accepted/rejected by the admin. This will help us introduce events based on attendees' choice and ultimately gaining audience.

3.2.8. Online Streaming

Attendees will be provided with an option to use 360 streaming which will use a 360-degree camera with VR Headset. It will help us to target introverts who do not want to participate in events physically and only want to experience events the way they want to.

3.2.9. Runtime notifications

Organizer will be able to generate notifications on the runtime related to the events. Attendees will receive that notification if they are registered for that specific event.

3.2.10. Send/receive messages

All the users will be able to send and receive messages in the specific rooms in which they are allowed. There will be three types of chat rooms i.e., private, general and event chat rooms.

3.3. Non-Functional Requirements

The non-functional requirements of the system are listed below.

3.3.1. Responsive Layouts

All our application's pages will be responsive which will remove the restriction of using our application on specific android phones.

3.3.2. Security Requirements

All our login pages will pass through authentication. All our passwords will be encrypted, and our payments will pass through a secured payment gateway.

3.3.3. Scalability

Our application will be able to adapt itself to increased usage as time progresses. As more and more events are introduced, attendees' data will also increase. Our app will be able to handle it without delay.

3.3.4. State Management

Our application will be able to return to its current state if it is interrupted by system services. e.g., if users receive a call, after returning, users will return to their previous state.

3.3.5. Availability

As our target is to publish our application over play store, so it will be available 24/7. So, users will be able use our application any time they want to.

4. Use Case Diagram

Figure 1 Use Case Diagram

5. High Level Use Case

5.1. Register / login

Table 5 Register/login High Level Use Case

Use case id	1
Use case name	Login/ Register
Actor	Attendees, Organizers, Admin
Type	Primary
Description	The user will register himself and then he will go through authentication to continue using the application.

5.2. Authentication

Table 6 Authentication High Level Use Case

Use case id	2
Use case name	Authentication
Actor	System
Type	Secondary
Description	The system will authenticate every login.

5.3. Update Profile

Table 7 Update Profile High Level Use Case

Use case id	3
Use case name	Update Profile
Actor	Attendees, Organizers, Admin
Type	Primary
Description	The user will update his profile.

5.4. Create Events

Table 8 Create Events High Level Use Case

Use case id	4
Use case name	Create Event
Actor	Admin
Type	Primary
Description	The admin will create an event.

5.5. Send / Receive Messages

Table 9 Send/Receive Messages High Level Use Case

Use case id	6
Use case name	Send/Receive Messages
Actor	Attendees, Organizers, Admin
Type	Secondary
Description	The user will be able to communicate with each other by sending or receiving messages.

5.6. View Online Stream

Table 10 View Online Stream High Level Use Case

Use case id	11
Use case name	View online stream
Actor	Attendee, organizers
Type	Primary
Description	Attendee will be able to get the online stream of the enrolled event if available.

5.7. Request For Event

Table 11 Request For Event High Level Use Case

Use case id	12
Use case name	Request for event
Actor	Attendee
Type	Primary
Description	Attendee will generate a request for an event to occur in which he/she is interested.

6. Expanded Use Case

6.1. Set Hierarchy

Table 12 Set Hierarchy Expanded Use Case

Use case id	5
Use case name	Set hierarchy
Actor	Admin
Description	Admin will set the hierarchy i.e., heads and co-heads for the event management.
Trigger	Admin clicks the set hierarchy button.
Pre-condition	Admin must be login.
Post condition	Organizers for the event has been added.
Normal flow	1. Admin will enter emails of organizers. 2. Entered emails will be verified. 3. Event organizers will be added.
Alternative flow	1. Invalid emails or user not found. 2. Try again.
Special Requirement	Internet availability
Frequency of use	High

6.2. Manage Sub-events

Table 13 Manage Sub-events Expanded Use Case

Use case id	7
Use case name	Manage sub-events
Actor	Organizers
Trigger	Organizers clicks the add sub-event button.
Pre-condition	Organizer must login using the email that admin has added has organizer.
Post condition	Sub-event will be created.
Normal flow	1. Organizer will add sub-event description. 2. Organizer will add pricing options if required. 3. Organizer will add polling options if required. 4. Organizer will add certificates if required.
Alternative flow	1. Enter valid information. 2. Try again.
Special Requirement	Internet availability
Frequency of use	High

6.3. Setup Online Stream

Table 14 Setup Online Stream Expanded Use Case

Use case id	7
Use case name	Setup online stream
Actor	Organizers
Trigger	Organizer clicks the online stream button
Pre-condition	Organizer must be login and has added a sub-event for the stream.
Post condition	Event will be streamed online.
Normal flow	1. Organizer will confirm the hardware connection. 2. Check the recordings. 3. Host the stream online.
Alternative flow	1. Hardware connection failed 2. Try again.
Special Requirement	Internet availability
Frequency of use	High

6.4. Generate Runtime Notifications

Table 15 Generate Runtime Notifications Expanded Use Case

Use case id	11
Use case name	Generate runtime notifications
Actor	Organizers
Trigger	Organizer clicks the generate notifications button.
Pre-condition	Organizer must login using the email that admin has added has organizer
Post condition	Notification will be sent to targeted users.
Normal flow	1. Organizer will add the notification description. 2. Select the users or group of users to whom he/she wants to send notification. 3. Notification sent.
Alternative flow	1. Error in sending notification. 2. Try again.
Special Requirement	Internet availability
Frequency of use	High

6.5. Enroll For Event

Table 16 Enroll For Event Expanded Use Case

Use case id	12
Use case name	Enroll
Actor	Attendee
Trigger	Organizer has clicked on the generate notifications.
Pre-condition	Attendee must be logged in.
Post condition	Attendee will be enrolled for the event.
Normal flow	1. Click on the Register. 2. It will redirect to registration page. 3. Attendee will enter enrollment information. 4. Attendee will make payment if required. 5. Attendee will vote if required.
Alternative flow	1. Enrollment process failed 2. Enter valid information. 3. Try again to enroll.
Special Requirement	Internet availability
Frequency of use	High

7. Domain Model

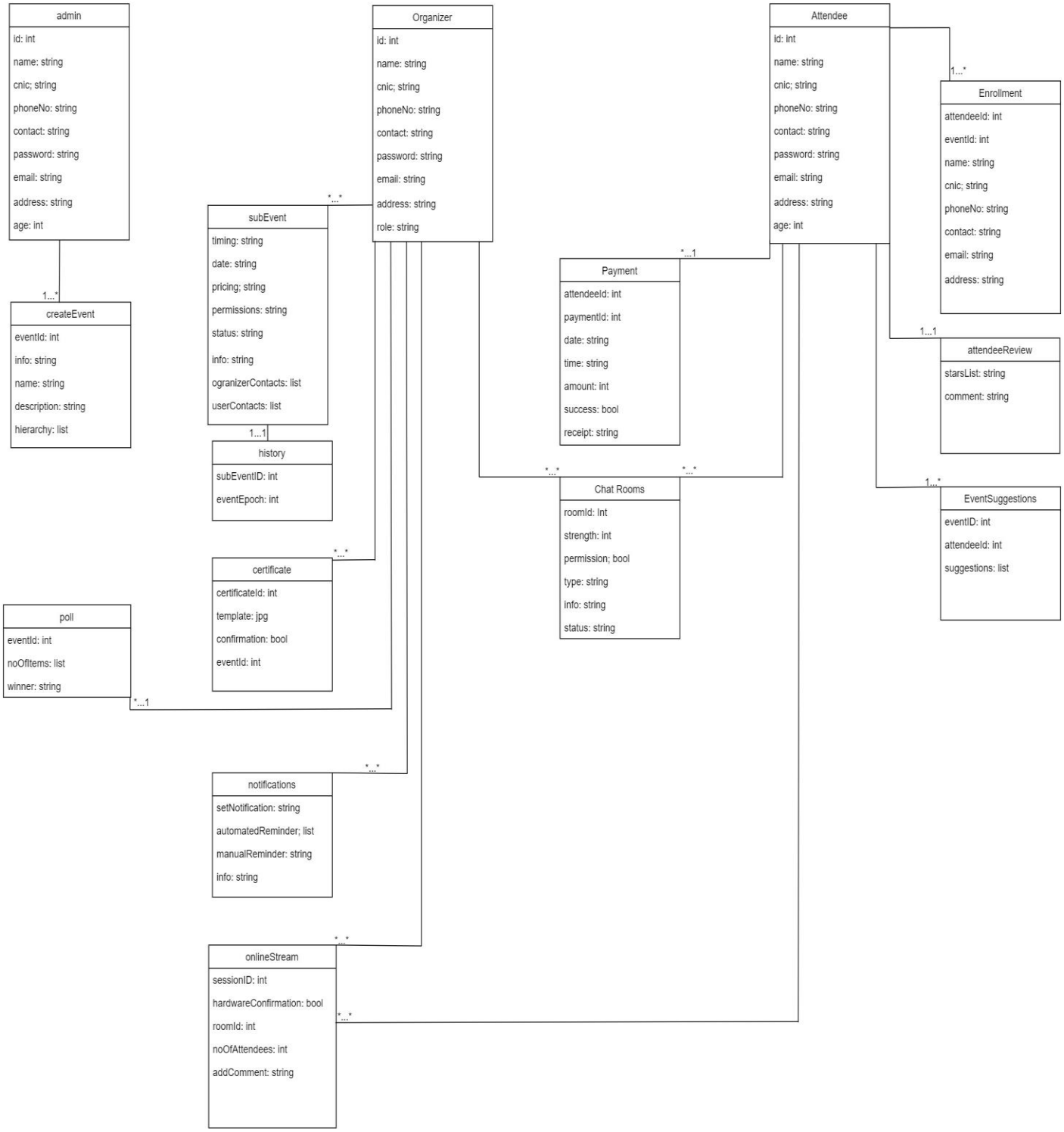

Figure 2 Domain Model

8. Class Diagram

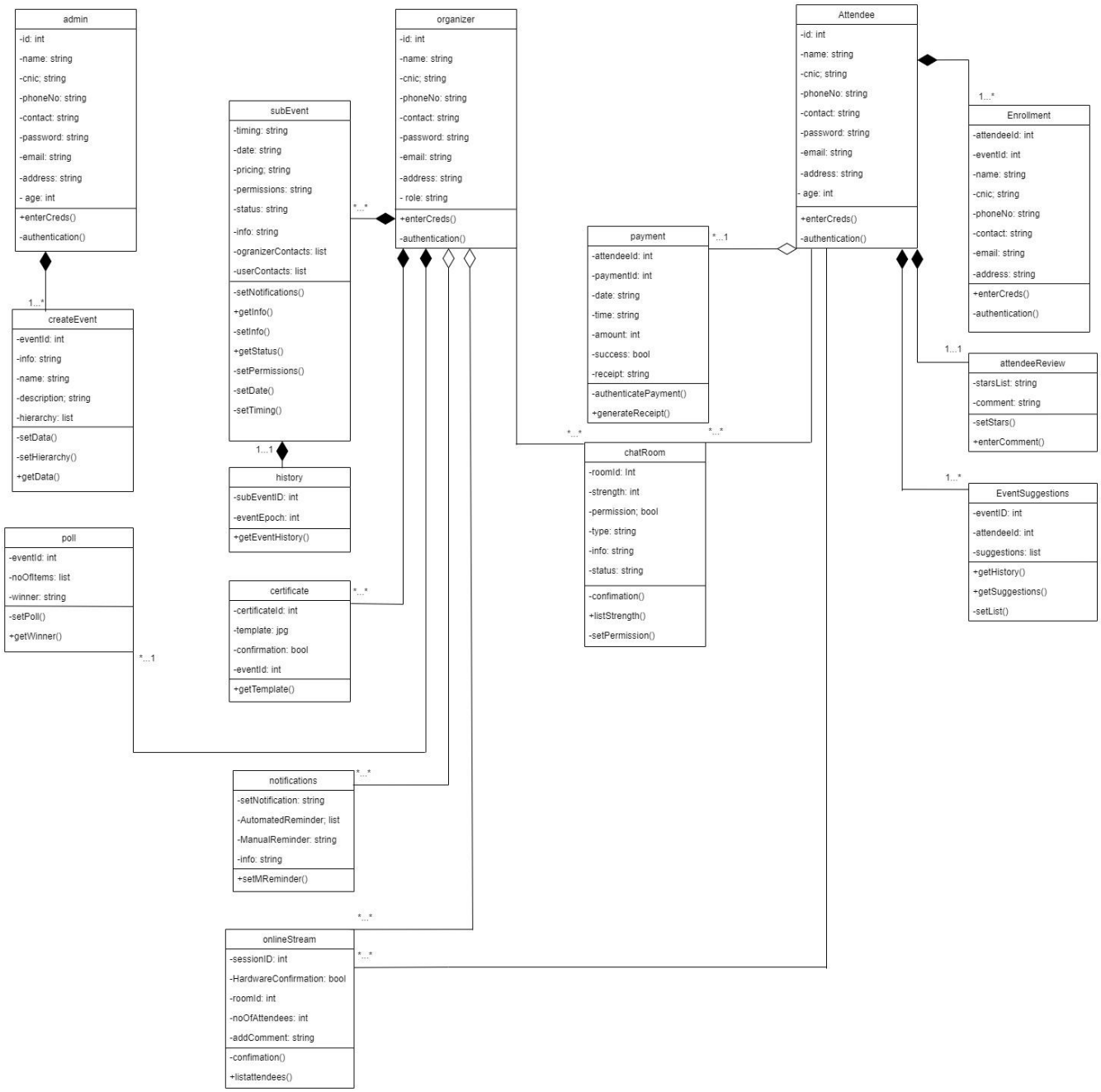

Figure 3 Class Diagram

9. Swimlane Diagram

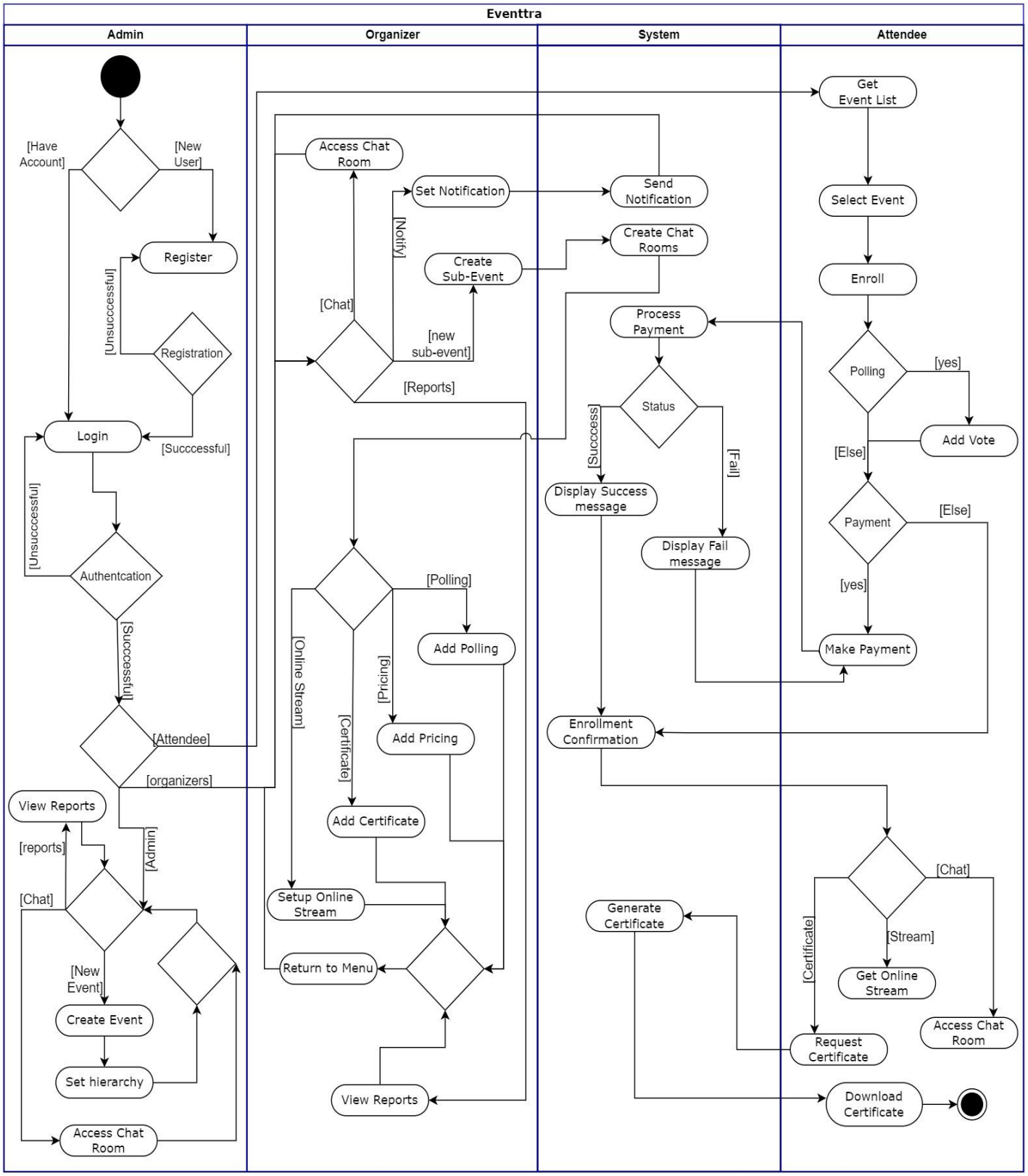

Figure 4 Activity Diagram

10. System sequence diagram

10.1 Set hierarchy

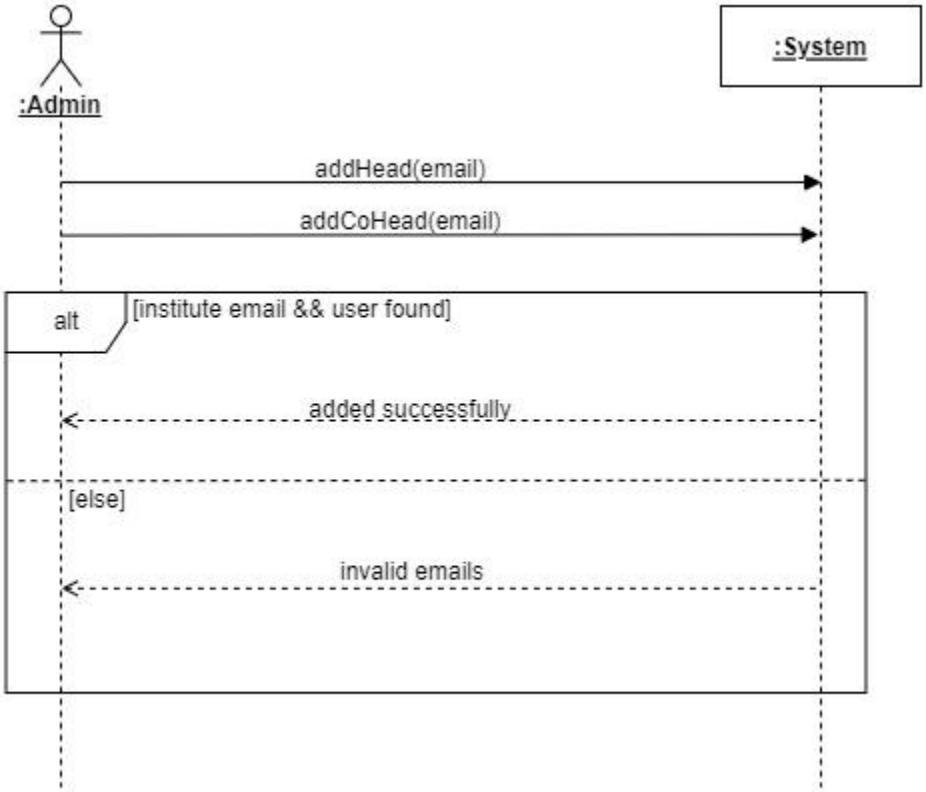

Figure 5 SSD set hierarchy

10.2 Manage Sub-events

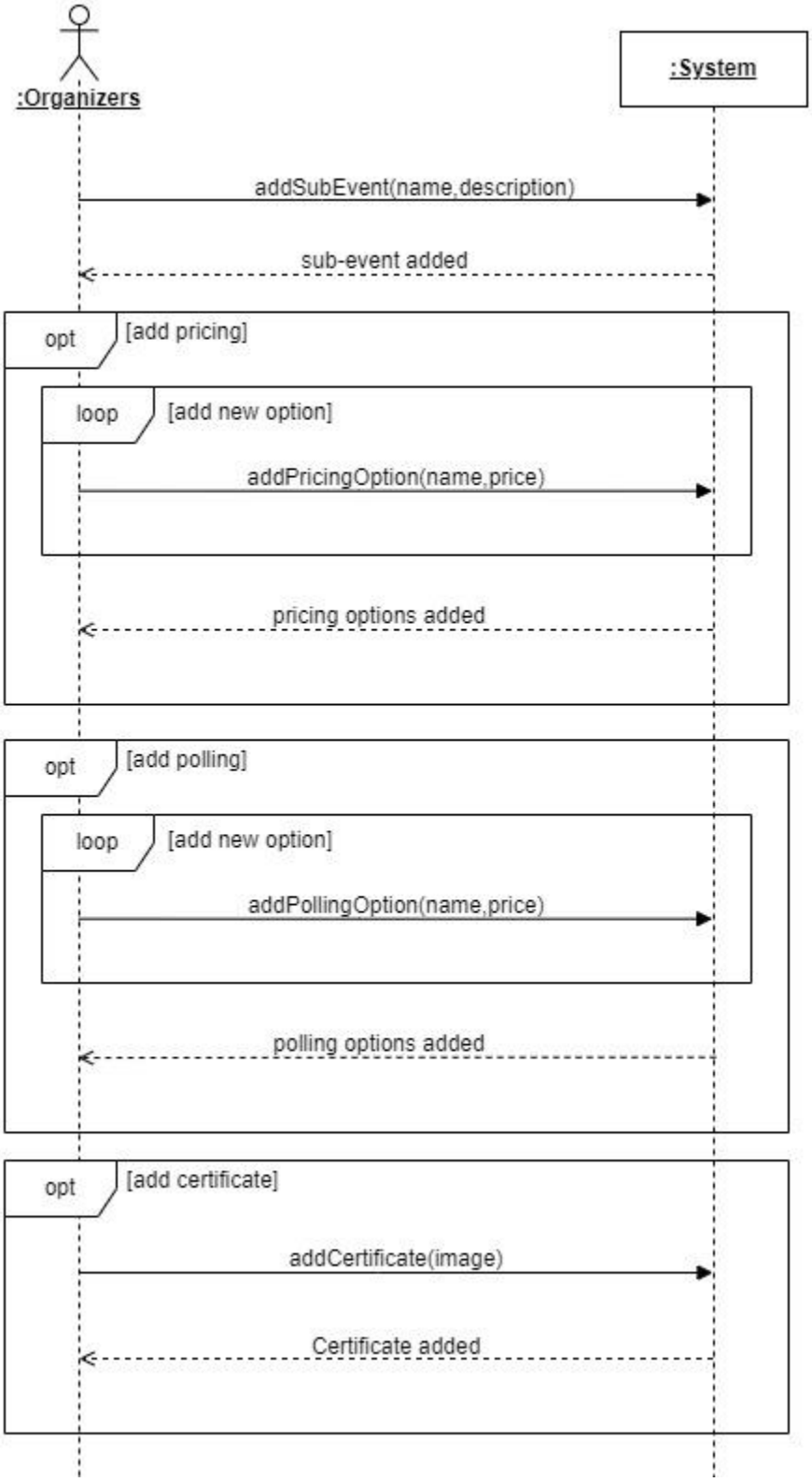

Figure 6 SSD Manage Sub-events

10.3 Setup Online Stream

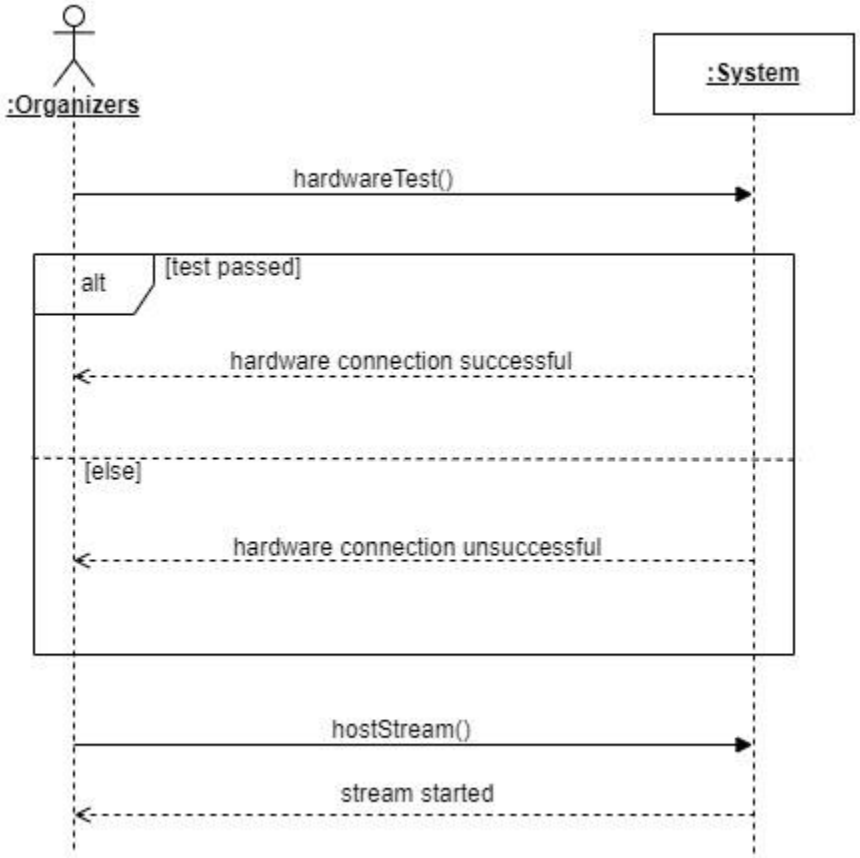

Figure 7 SSD Setup Online Stream

10.4 Generate Runtime Notifications

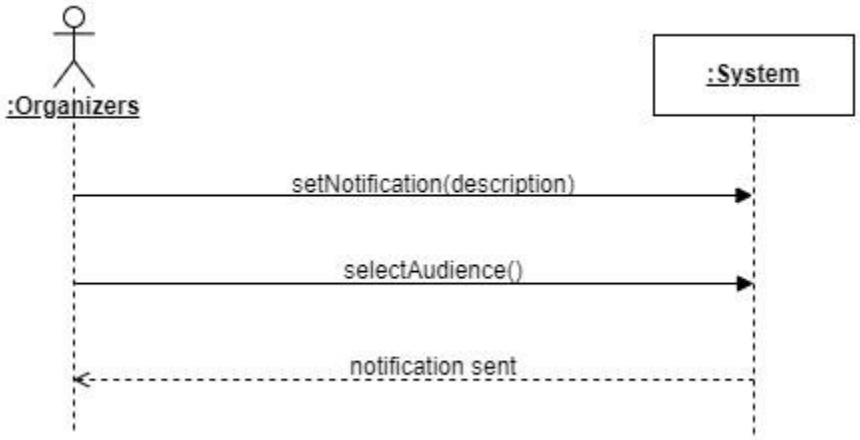

Figure 8 SSD Generate Runtime Notification

10.5 Enroll For Event

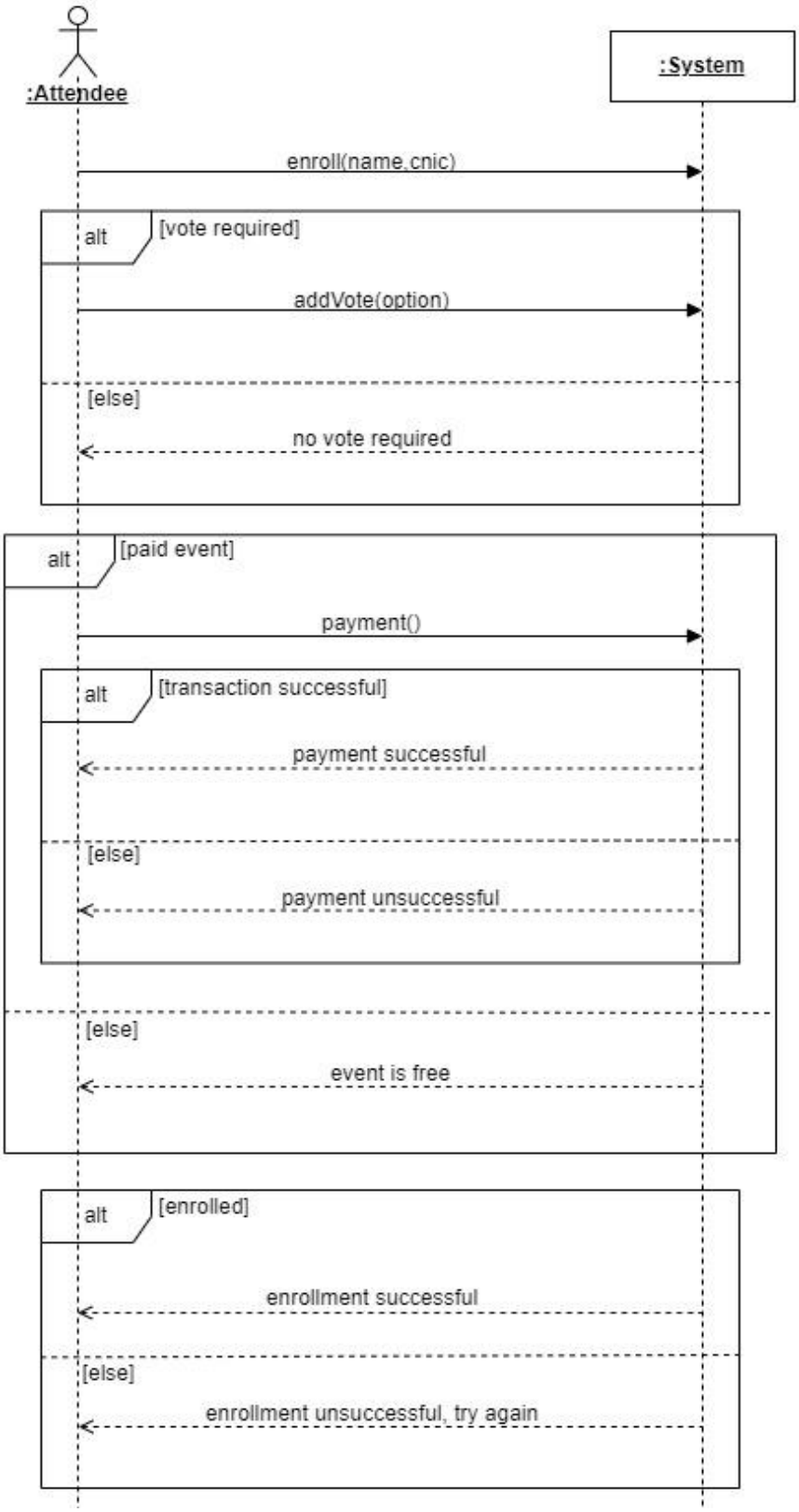

Figure 9 SSD Enroll For Event

11. Sequence Diagram

11.1 Admin

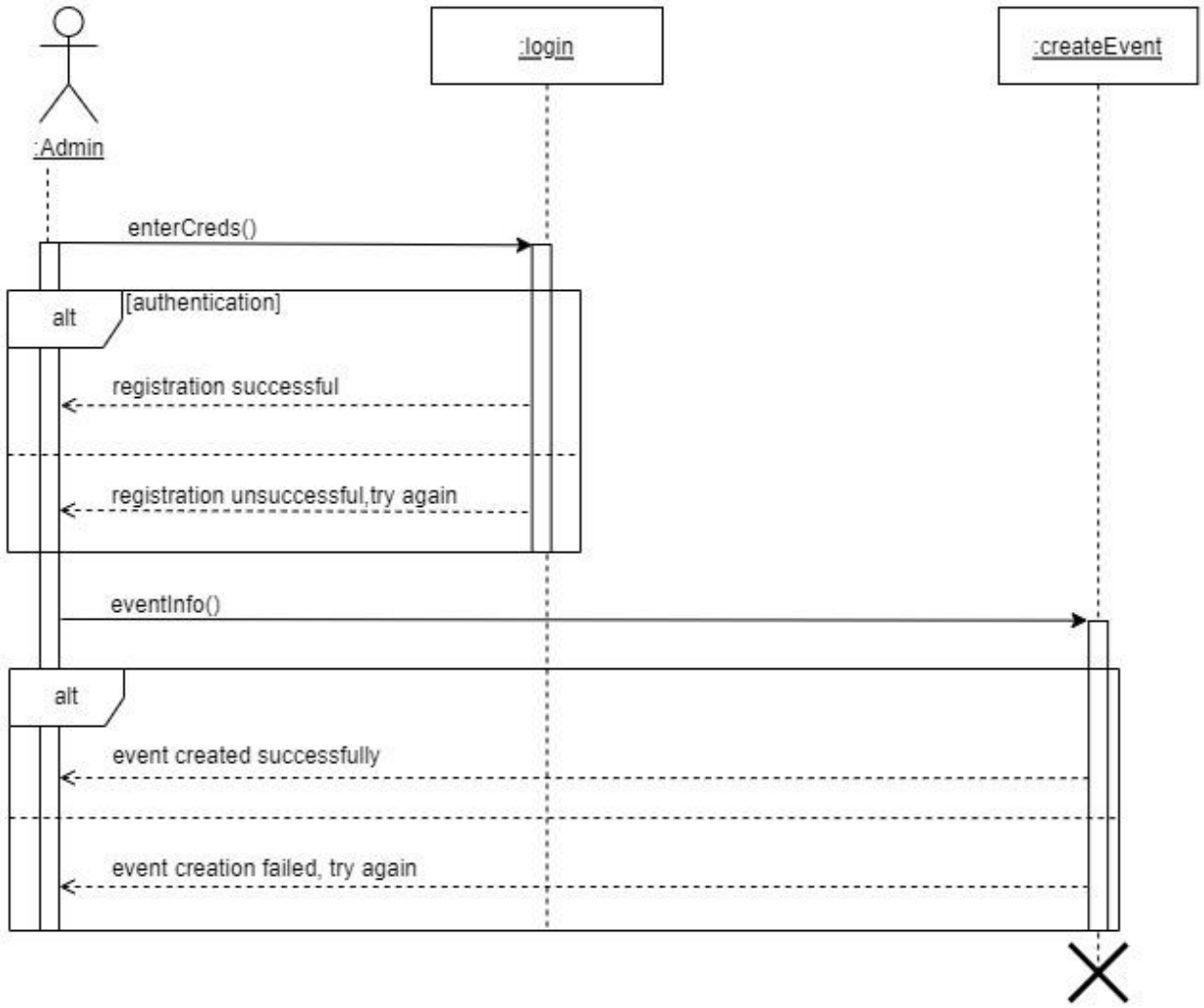

Figure 10 Admin Sequence Diagram

11.2 Organizers

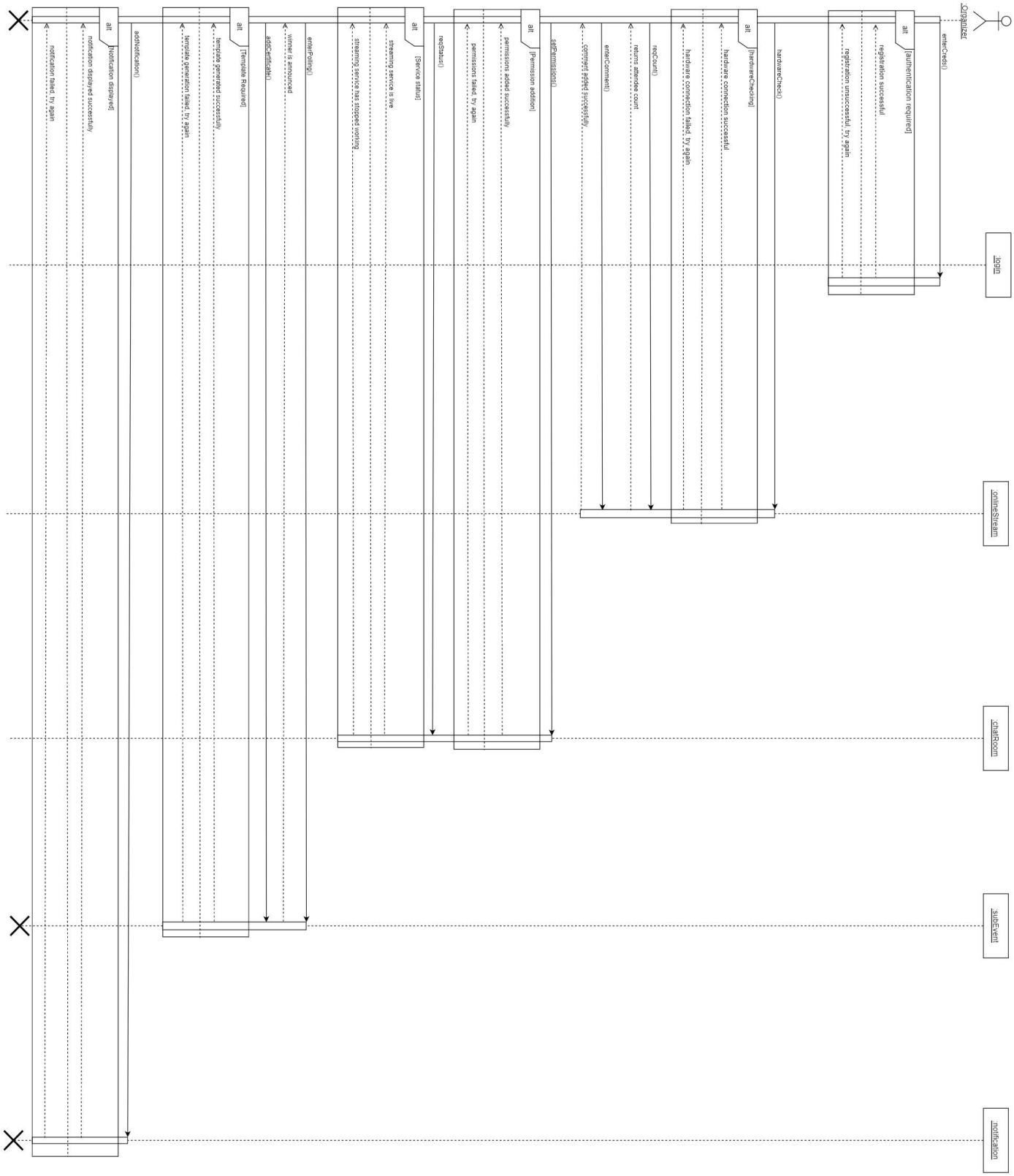

Figure 11 Organizer Sequence Diagram

11.3 Attendee

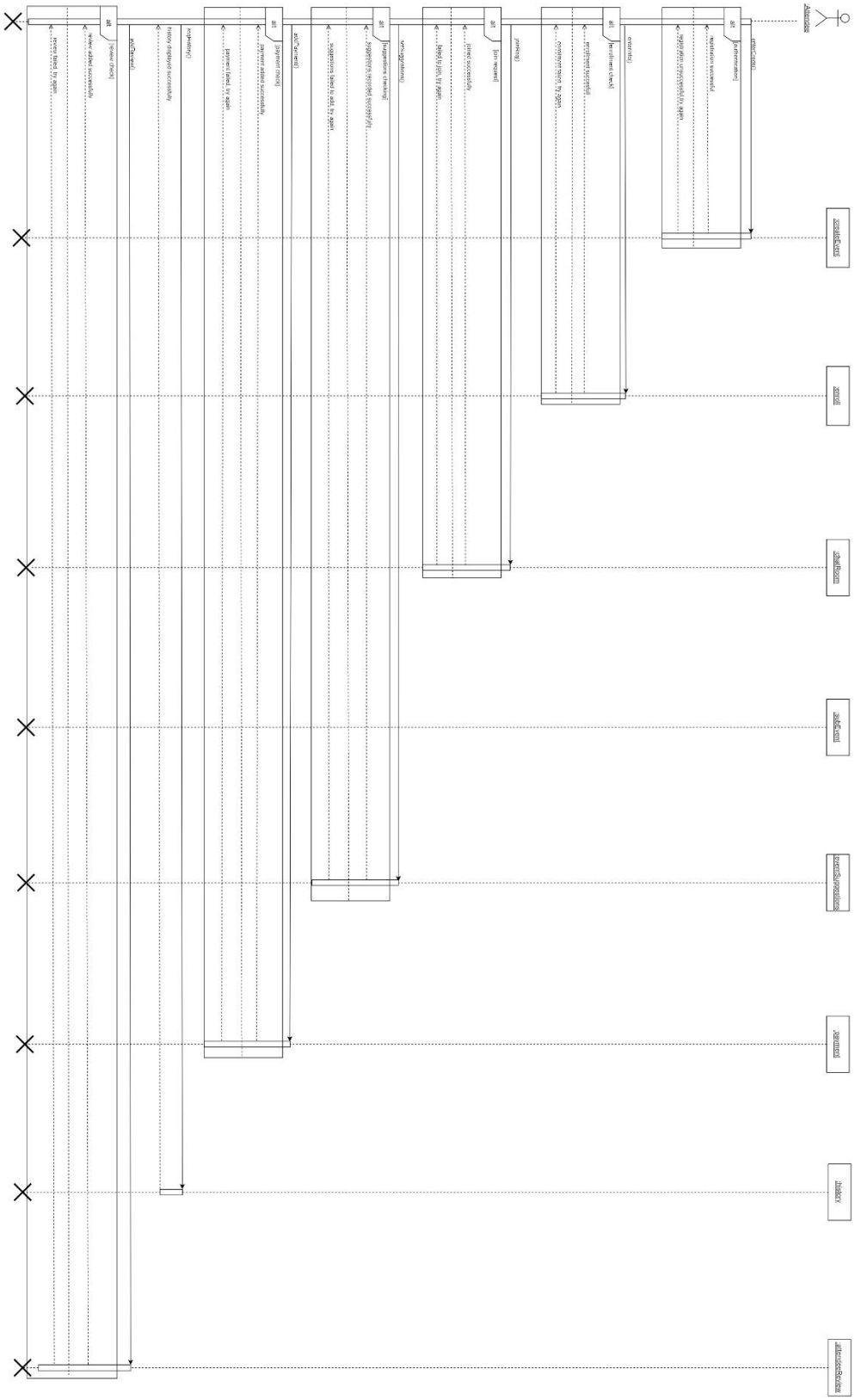

Figure 12 Admin Sequence Diagram

12. Data Flow Diagrams

12.1. Level 0 Data Flow Diagram

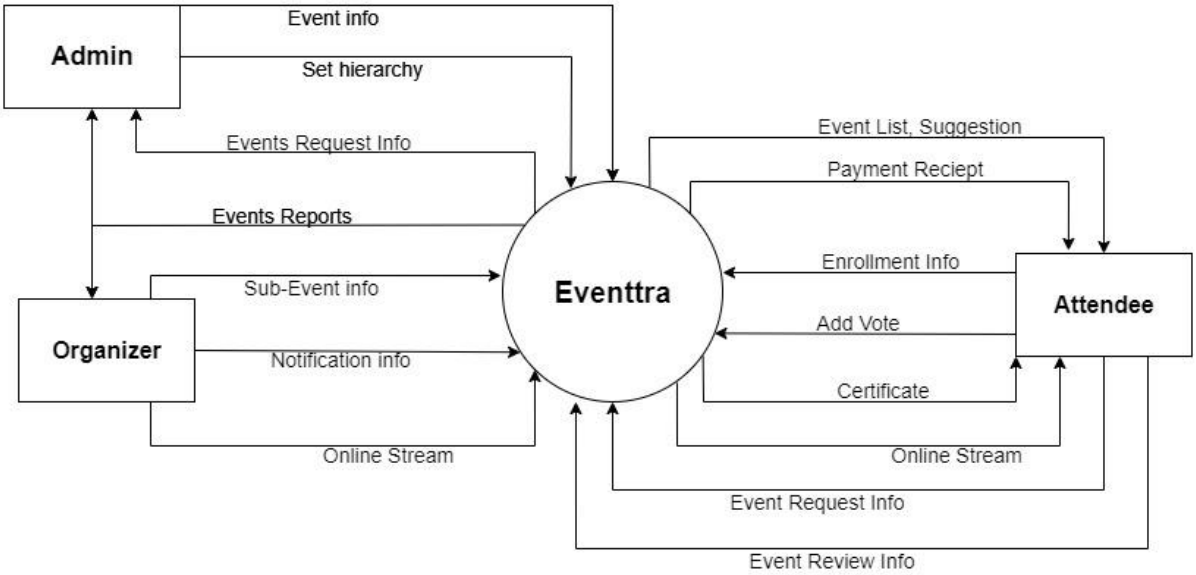

Figure 13 Level 0 DFD

12.2. Level 1 Data Flow Diagram

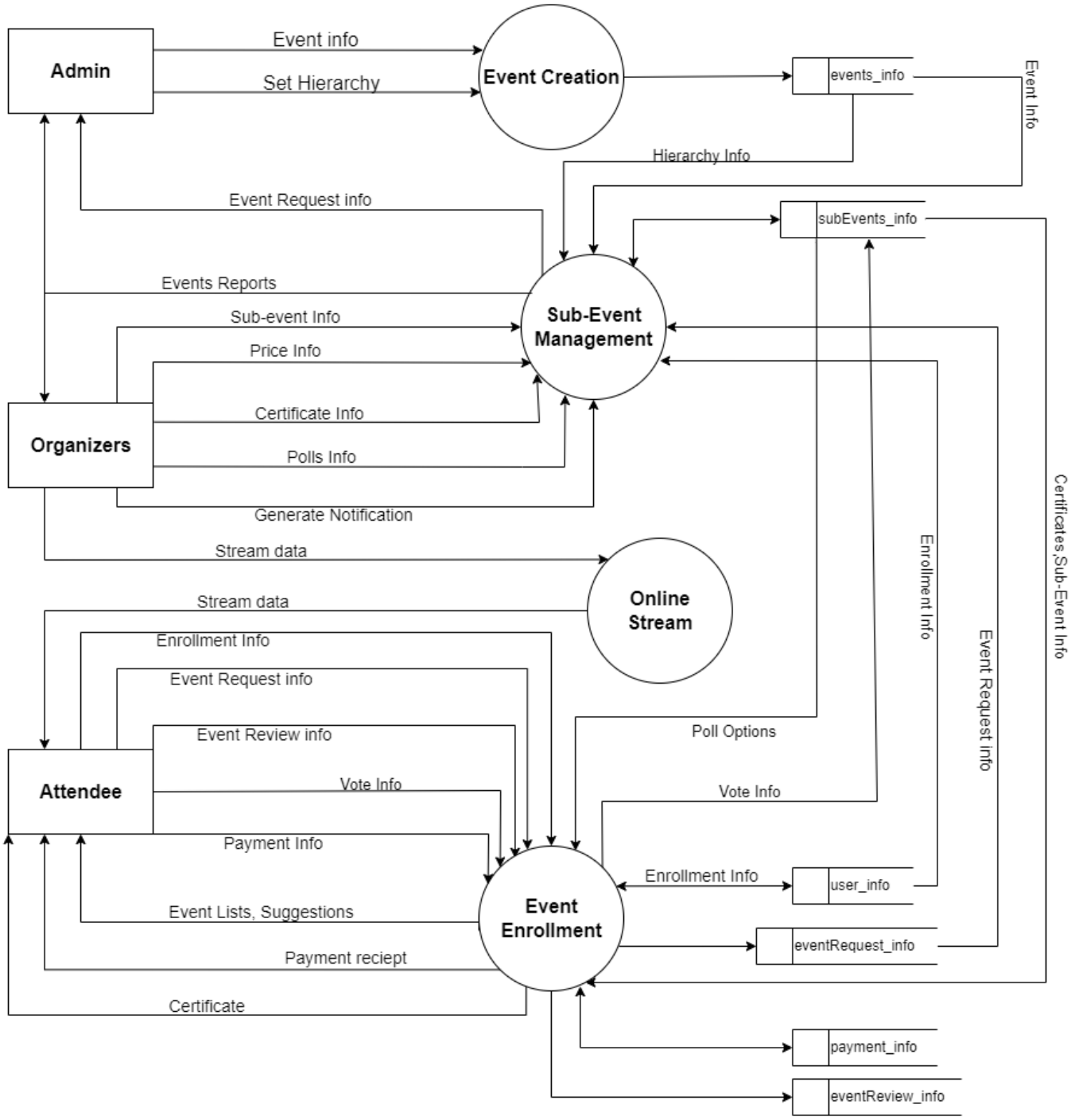

Figure 14 Level 1 DFD

12.3. Level 2 Data Flow Diagram

12.3.1. Event Creation:

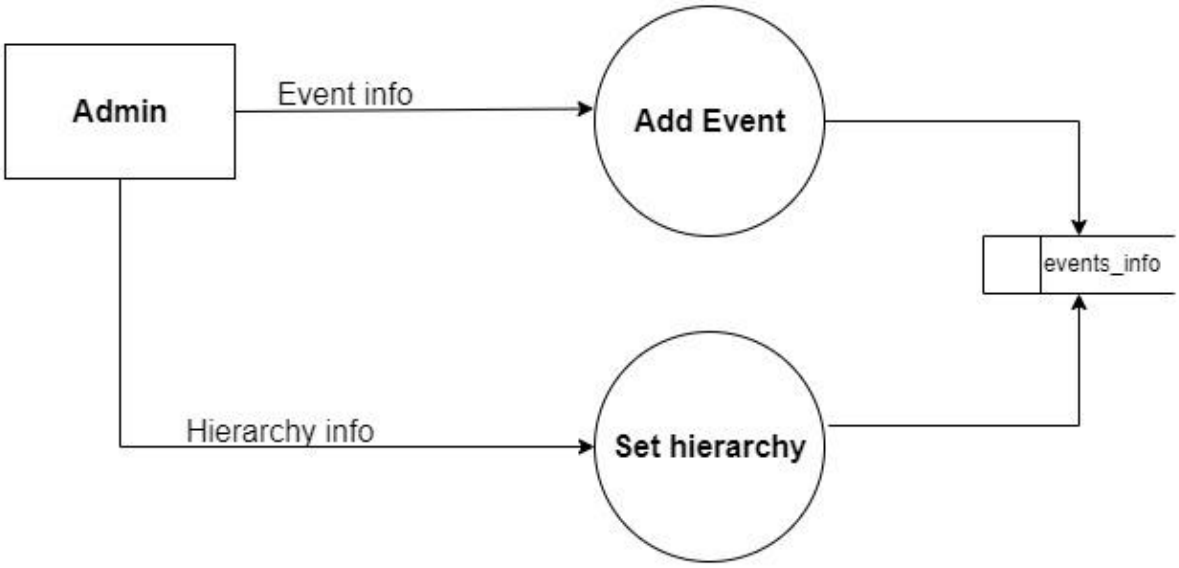

Figure 15 Level 3 Event Creation DFD

12.3.2. Sub-Event Management:

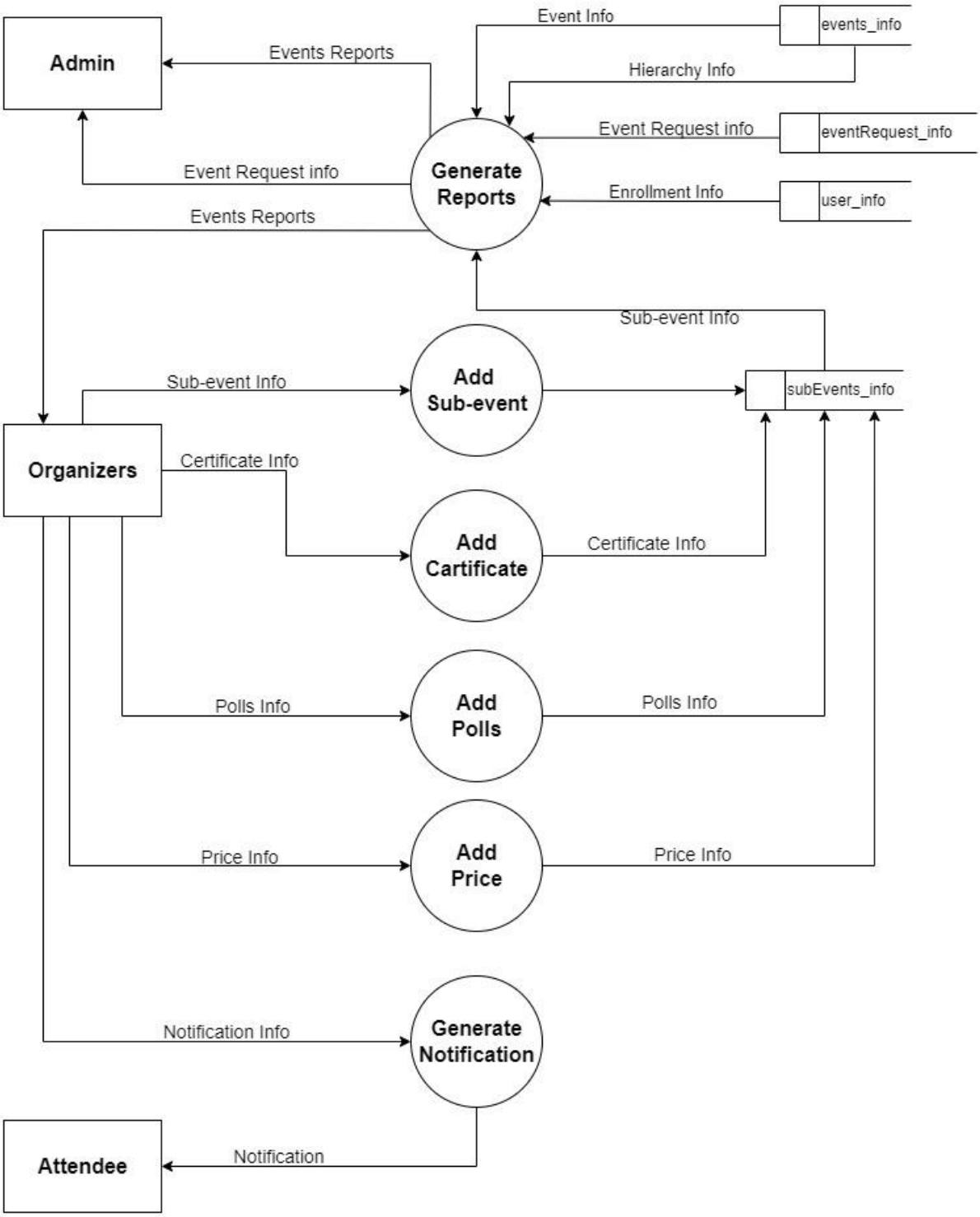

Figure 16 Level 3 Sub-Event Management DFD

12.3.3. Online Stream:

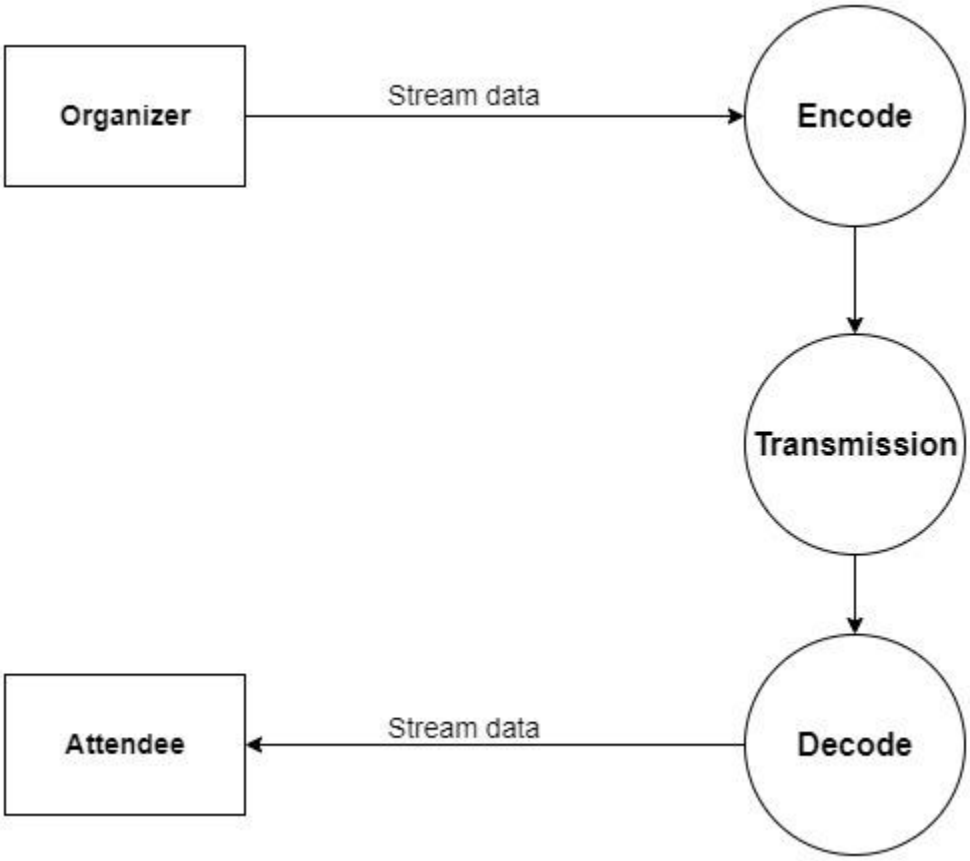

Figure 17 Level 3 Online Stream DFD

12.3.4. Event Enrollment:

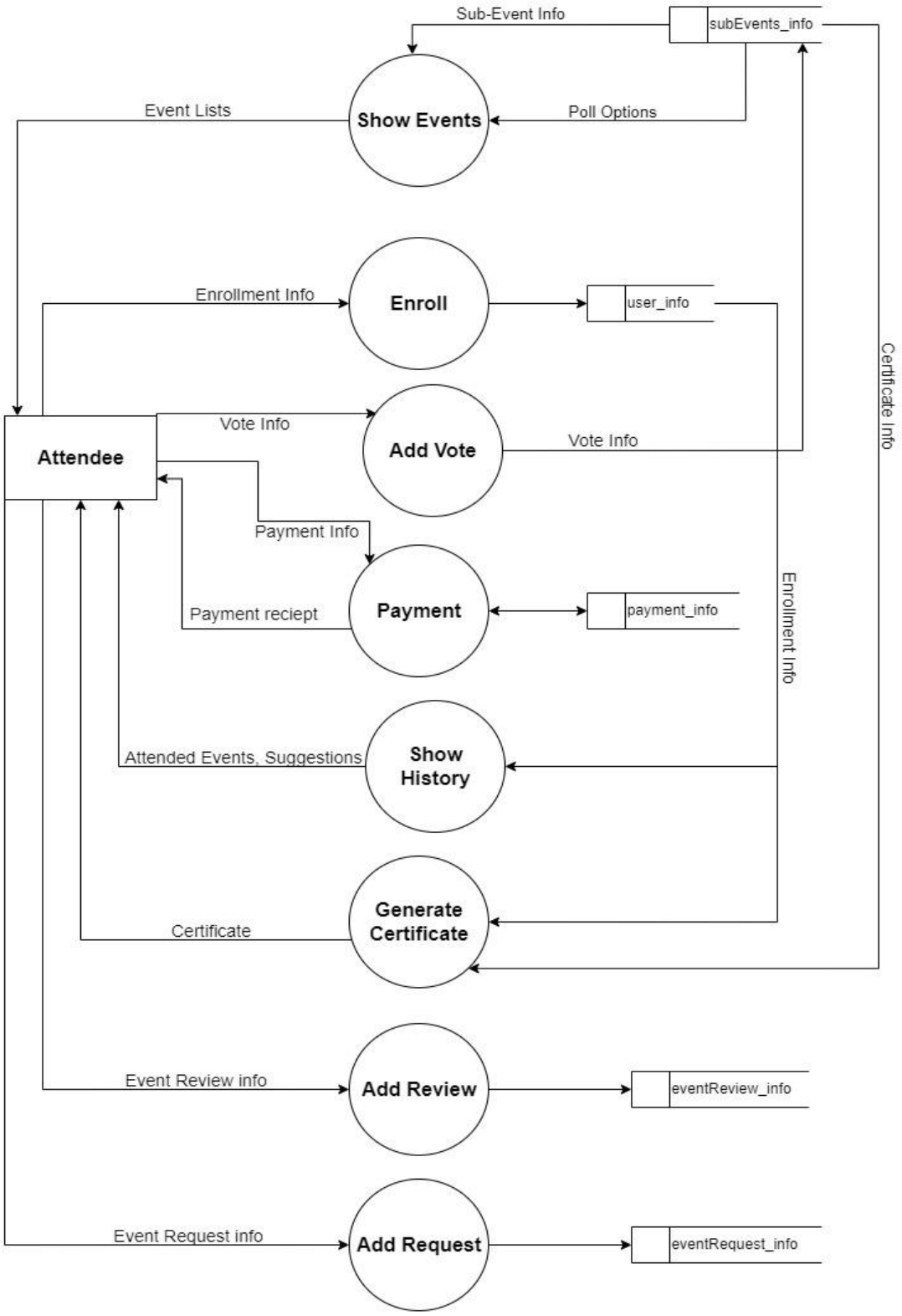

Figure 18 Level 3 Event Enrollment DFD

13. Package Diagram

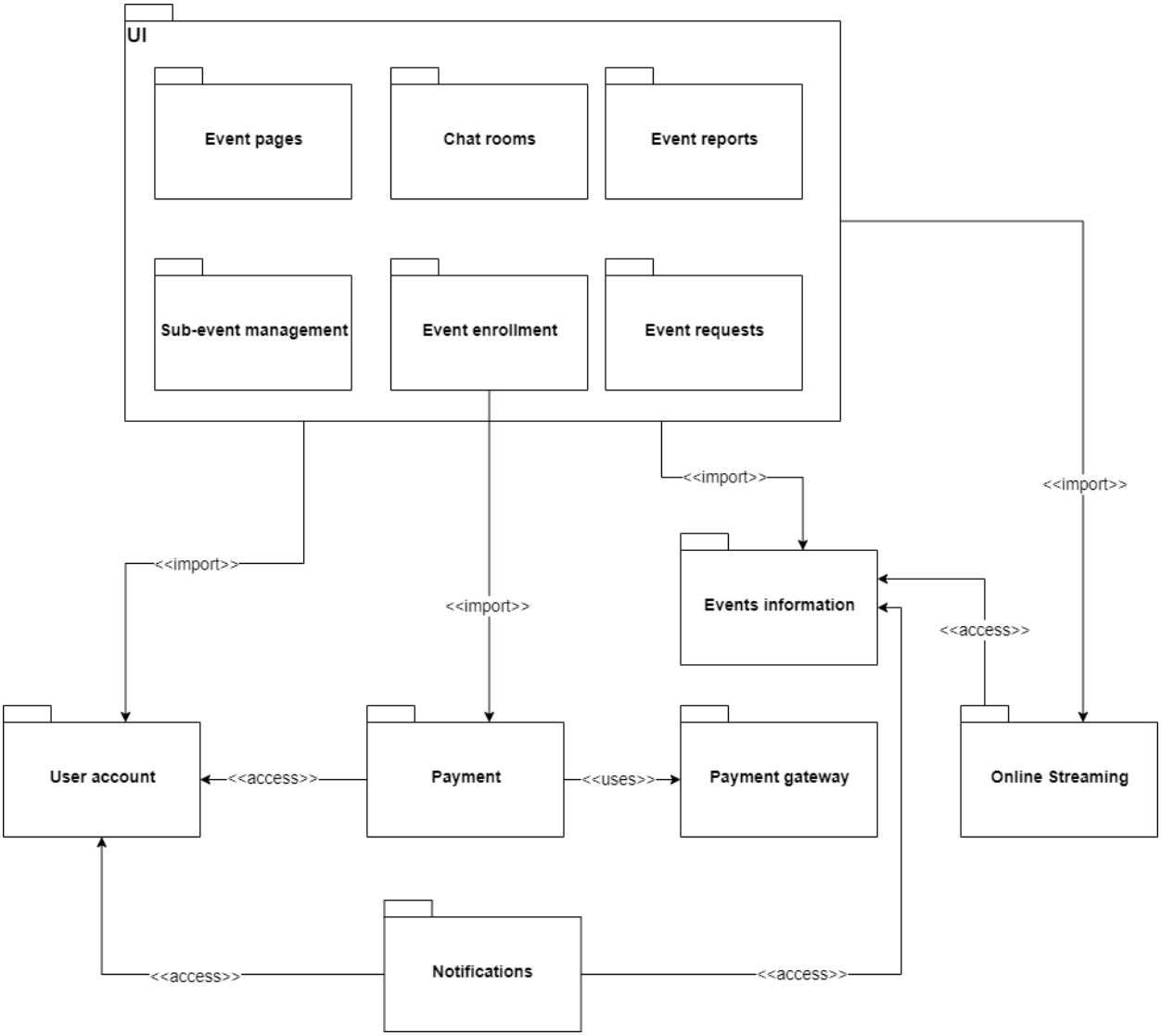

Figure 19 Package Diagram

14. Database Schema

Figure 20 Database Schema

15. Comparison

	Eventtra	Event hub	Evento
Search filtering	Yes	Yes	No
Event polling	Yes	No	No
Event request	Yes	No	No
Hierarchy maintained	Yes	Yes	Yes
Online Streaming	Yes	No	No
Event History	Yes	Yes	Yes
Runtime notifications	Yes	No	No
Pricing discounts	Yes	No	No
Online certificates	Yes	No	No
Event suggestions	Yes	Yes	No
Prohibited categories	No	Yes	No

Table 17 Comparison

16. Unit Test Plan

16.1 Introduction

The purpose of this Unit Test Plan is to document the testing approach and test cases for the Eventtra. Eventtra provides management services including a description page for events, hierarchy management and chat rooms for better communication. Attendees can experience events using VR Headset streamed on 360-degree camera. At the end attendees receives certificates for their performance.

16.2 Test Environment

- Android Studio
- Junit

16.3 Test Approach

Unit testing of modules is implemented which:

- Tests modules by providing Inputs that truthify
- Also tests modules by providing Inputs that falsify

16.4 Test Reporting

Test results are reported in the following format:

- Test case ID
- Input values
- Expected output
- Actual output
- Pass/Fail status

16.5 Test Deliverables

The following deliverables will be produced from the unit testing:

- Unit Test Plan
- Unit Test Report

16.6 Test Management

The developers will be responsible for conducting and verifying the unit tests i.e., writing the unit tests, executing them, verifying the test results and reporting any issues.

16.7 Unit Test Report

16.7.1 Module: addEventdetails.java				
Unit: testValidDate1()				
Test case ID	**Input Values**	**Expected Output**	**Actual Output**	**Pass/Fail Status**
00001	startYear = 2023, startMonth = 3, startDay = 29, endYear = 2023, endMonth = 3, endDay = 30;	True	True	Passed
00002	startYear = 2023, startMonth = 4, startDay = 15, endYear = 2023, endMonth = 5, endDay = 20;	True	True	Passed
Unit: Unit: testInvalidDate1				
00003	startYear = 2023, startMonth = 3, startDay = 29, endYear = 2023, endMonth = 3, endDay = 30;	False	False	Passed
00004	startYear = 2023, startMonth = 5, startDay = 2, endYear = 2023, endMonth = 3, endDay = 30;	False	False	Passed

Table 18 addEventDetailsTest

16.7.2 Module: adminEventRequests.java				
Unit: dataValid1()				
Test case ID	**Input Values**	**Expected Output**	**Actual Output**	**Pass/Fail Status**
00005	test = EventRequestModel("1", "Daira","This is Daira", "Reject" , "11") stub = EventRequestModel("1", "Daira","This is Daira", "Reject" , "11")	("1","Daira","This is Daira", "Reject" , "11")	("1","Daira","This is Daira", "Reject" , "11")	Passed
00006	test = EventRequestModel("2", "Daira2","This is Daira 2", "Accept" , "12") stub = EventRequestModel ("2", "Daira2","This is Daira 2", "Accept" , "12")	("2", "Daira2","This is Daira 2", "Accept" , "12")	("2", "Daira2","This is Daira 2", "Accept" , "12")	Passed
Unit: dataInvalid1()				
00007	test = EventRequestModel("1", "Daira2","This is Daira", "Reject" , "11") stub = EventRequestModel("1", "Daira","This is Daira", "Reject" , "11")	("1", "Daira2","This is Daira", "Reject" , "11")	("1","Daira","This is Daira", "Reject" , "11")	Passed
00008	test = EventRequestModel("1", "Daira3","This is Daira", "Reject" , "11") stub = EventRequestModel("1", "Daira","This is Daira", "Reject" , "11")	("1", "Daira3","This is Daira", "Reject" , "11")	("1","Daira","This is Daira", "Reject" , "11")	Passed

Table 19 adminEventRequestsTests

16.7.3 Module: MyEvent.java				
Unit: MyEventValid1()				
Test case ID	**Input Values**	**Expected Output**	**Actual Output**	**Pass/Fail Status**
00009	test = MyEvent("1","Event 1","This is Event 1","2-3-2023","2-3-2023",myMap,uri2) stub = MyEvent("1","Event 1","This is Event 1","2-3-2023","2-3-2023",myMap,uri2)	("1","Event 1","This is Event 1","2-3-2023","2-3-2023",myMap,uri2)	("1","Event 1","This is Event 1","2-3-2023","2-3-2023",myMap,uri2)	Passed
00010	test = MyEvent("2","Event 2","This is Event 2","3-3-2023","4-3-2023",myMap,uri2) stub = MyEvent("2","Event 2","This is Event 2","3-3-2023","4-3-2023",myMap,uri2)	("2","Event 2","This is Event 2","3-3-2023","4-3-2023",myMap,uri2)	("2","Event 2","This is Event 2","3-3-2023","4-3-2023",myMap,uri2)	Passed
Unit: MyEventInvalid1()				
00011	test = MyEvent("2","Event 3","This is Event 3","2-3-2023","2-3-2023",myMap,uri2) stub = MyEvent("1","Event 1","This is Event 1","2-3-2023","2-3-2023",myMap,uri2)	("2","Event 3","This is Event 3","2-3-2023","2-3-2023",myMap,uri2)	("1","Event 1","This is Event 1","2-3-2023","2-3-2023",myMap,uri2)	Passed
00012	test = MyEvent("2","Event 4","This is Event 4","2-3-2023","2-3-2023",myMap,uri2); stub = MyEvent("1","Event 1","This is Event 1","2-3-2023","2-3-2023",myMap,uri2	("2","Event 4","This is Event 4","2-3-2023","2-3-2023",myMap,uri2);	("1","Event 1","This is Event 1","2-3-2023","2-3-2023",myMap,uri2	Passed

Table 20 MyEventTest

16.7.4Module: MyUser.java				
Unit: MyUserValid1()				
Test case ID	**Input Values**	**Expected Output**	**Actual Output**	**Pass/Fail Status**
00013	test=MyUser ("Awais","Shahid","awaishahid.147@gmail.com","03061601266","attendee","1",uri2,"token 1"); stub=MyUser("Awais","Shahid","awaishahid.147@gmail.com","03061601266","attendee","1",uri2,"token 1")	stub(1)	stub(1)	Passed

00014	test=MyUser("Muhammad","Saad","m.saad.147@gmail.com","03331111111","attendee","1",uri2,"token 1");
	
	stub=MyUser("Muhammad","Saad","m.saad.147@gmail.com","03331111111","attendee","1",uri2,"token 1");
		stub(2)	stub(2)	Passed

Unit: MyUserInValid1 ()				
00015	test= MyUser ("test1","test1","awaishahid.147@gmail.com","03061601266","attendee","1",uri2,"token 1") stub=MyUser("Awais","Shahid","awaishahid.147@gmail.com","03061601266","attendee","1",uri2,"token 1")	stub(1)	MyUser ()	Passed
00016	test= MyUser("test2","test2","awaishahid.147@gmail.com","03061601266","attendee","1",uri2,"token 1") stub=MyUser("Awais","Shahid","awaishahid.147@gmail.com","03061601266","attendee","1",uri2,"token 1")	stub(1)	MyUser ()	Passed

Table 21 MyUserTest

16.7.5 Module: PaymentInfo.java				
Unit: PaymentValid1()				
Test case ID	**Input Values**	**Expected Output**	**Actual Output**	**Pass/Fail Status**
00017	test=PaymentInfo("1","2","Awais",false,100,"Fatima","12345678901","1","Dota 2",12345) stub=PaymentInfo("1","2","Awais",false,100,"Fatima","12345678901","1","Dota 2",12345)	stub(1)	stub(1)	Passed
00018	test=PaymentInfo("2","3","tester",true,200,"Awais","12355578901","2","Cricket",13345) stub=PaymentInfo("2","3","tester",true,200,"Awais","12355578901","2","Cricket",13345)	stub(2)	stub(2)	Passed
Unit: PaymentInvalid1()				
00019	test =PaymentInfo("4","4","tester 3",false,100,"Awais","12345678901","1","Football",12345); stub=PaymentInfo("1","2","Awais",false,100,"Fatima","12345678901","1","Dota 2",12345)	stub(1)	MyEvent()	Passed
00020	test =PaymentInfo("5","5","tester 3",false,100,"Awais","12345678901","1","Football",12345); stub=PaymentInfo("1","2","Awais",false,100,"Fatima","12345678901","1","Dota 2",12345)	stub(1)	MyEvent()	Passed

Table 22 paymentInfoTest

16.7.6 Module: subEventRequests.java				
Unit: subEventRequestsValid1()				
Test case ID	**Input Values**	**Expected Output**	**Actual Output**	**Pass/Fail Status**
00021	test=subEventsModel("1","Dota 2","This is Esports","100","Awais", uri2,"1","2/3/2023","16:16:16",5,false) stub=subEventsModel("1","Dota 2","This is Esports","100","Awais", uri2,"1","2/3/2023","16:16:16",5,false))	stub(1)	stub(1)	Passed
00022	test=subEventsModel("2","Cricket","This is Cricket","100","Awais", uri2,"1","2/3/2023","16:16:16",5,true) stub= subEventsModel("2","Cricket","This is Cricket","100","Awais", uri2,"1","2/3/2023","16:16:16",5,true)	stub(2)	stub(2)	Passed
Unit: subEventRequestsNotValid1()				
00023	test = subEventsModel("1","Dota 3","This is not released","100","Awais", uri2,"1","2/3/2023","16:16:16",5,false) stub=subEventsModel("1","Dota 2","This is Esports","100","Awais", uri2,"1","2/3/2023","16:16:16",5,false))	stub(1)	subEventsModel()	Passed
00024	test = subEventsModel("1","Tekken 7","This is not released","100","Awais", uri2,"1","2/3/2023","16:16:16",5,false) stub=subEventsModel("1","Dota 2","This is Esports","100","Awais", uri2,"1","2/3/2023","16:16:16",5,false))	stub(1)	subEventsModel()	Passed

Table 23 subEventRequests